

이력서

김선준 1993년생 | 남성 | 경력 4년차 | cyon13022@gmail.com

"데이터가 흐르는 파이프라인을 만들고, 그 위에서 Agent를 구현하는 엔지니어"

UNIST에서 추천 시스템을 연구했고, 졸업 후 벡터 검색 기반 서비스부터 vLLM 서빙, Agentic RAG, Reflexion Agent까지 데이터 수집-전처리-벡터화-검색-생성-서빙 전 구간과 연관된 프로젝트들을 수행해 왔음. 석사 때 다룬 임베딩-검색 구조가 이후 pgvector, Milvus 프로젝트에 그대로 이어졌고, 모델 하나 띄우는 게 아니라 파이프라인 전체를 직접 설계-구현하는 방식으로 일해 왔음.

[개발 전략]

- 파이프라인 전 구간에 대한 고려:** 전처리에서 병목이 터지거나 후처리에서 포맷이 잘못 나오면 모델이 아무리 좋아도 파이프라인 전체가 무너짐. 데이터 수집, 임베딩, 검색, 서빙 — 어느 한 구간이 빠져도 서비스가 안 되기 때문에, 설계 시 모든 단계를 고려함.
- 데이터 품질이 모델 성능을 결정:** 같은 RAG 구조라도 청크를 어떻게 자르고 어떤 메타 정보를 붙이느냐에 따라 검색 정확도가 달라짐. RAG 시스템에서 100자 이하 청크 필터링, 특수문자 제거, KoNLPy 기반 토큰 검증을 적용해 벡터 품질을 개선한 것이 대표적.
- 복잡한 태스크는 단순한 태스크로 분해부터:** 큰 문제를 한 번에 풀면 어디서 막혔는지 파악이 안 됨. RAG 시스템을 만들 때도 전처리, 임베딩, 검색, 리랭킹, 생성, 안전 검증을 각각 독립된 단위로 쪼개서 하나씩 검증했고, vLLM API도 요약·분류·태깅을 별도 에이전트로 분리한 뒤 LangGraph로 흐름을 엮음. 단위가 작아야 디버깅이 빠르고 설계도 유연해짐.
- 적절한 레퍼런스 참고로 불필요한 시행착오 줄이기:** 논문, 공식 문서, 검증된 구현체를 먼저 조사함. RAG에서 Dense+BM25 하이브리드 검색을 택한 것도, Reflexion Agent에서 ReAct 논문의 루프 구조를 가져온 것도 레퍼런스를 먼저 조사한 결과. 남이 이미 부딪힌 벽을 똑같이 부딪힐 이유가 없고, 그 시간을 아껴야 레퍼런스가 없는 문제에 집중할 수 있음.
- 연구 자료와 히스토리 정리(Documentation):** 조사 내용, 시도한 접근법, 실패 원인을 그때그때 기록으로 남김. Notion으로 기술 조사와 설계 판단 근거를 기록하고, Wikidocs로 연구자료를 정리해 저서로 만들기도 함. 이 기록이 이후 프로젝트에서 의사결정 속도를 높여주는 자산이 됨.

학력

최종학력: 대학원 석사졸업 2021.03 ~ 2023.02 | 졸업 UNIST(울산과학기술원) (울산) 산업공학과 논문 | Kim, Sunjun. Personal recommender system via convolutional autoencoder with conditioning augmentation. UNIST, 2023. Convolutional Autoencoder로 사용자 선호를 잠재 벡터로 표현하고, 벡터 유사도로 아이템을 추천하는 연구. 이때 다룬 임베딩-검색 파이프라인이 졸업 후 pgvector, Milvus 프로젝트의 기반이 됨. [논문 링크](#)

Skills

- Language: Python
- LLM/AI: vLLM, LangGraph, LangChain, LiteLLM, OpenAI API, HuggingFace
- Vector DB: Milvus, Pgvector
- Database: PostgreSQL, Oracle, MySQL
- Backend: FastAPI, Uvicorn

- Embedding: BGE-M3, BGE-Reranker, Sentence-Transformers
- Infra: Docker, Docker Compose
- Protocol: MCP (Model Context Protocol), SSE, JSON-RPC

경력

[주요 경력 및 프로젝트 요약]

(주)플랜티넷 (Plantynet) | AI 서비스 개발 (2024.10 – 현재)

- vLLM 기반 텍스트 처리 API 서비스: in-process vLLM 서버 + LangGraph 에이전트 기반 요약·분류·태깅 6종 API 통합
- Milvus 기반 Agentic RAG 시스템: Dense(BGE-M3)+BM25 하이브리드 검색, BGE-Reranker 재정렬, LlamaGuard 안전 검증, SSE 스트리밍 챗봇
- ReAct + Reflexion 자기 개선형 AI 에이전트: 실패 원인 분석 → 교훈 저장 → 재활용하는 자기 개선 Agent, 17개 도구 통합

(주)AIO2O | AI 서비스 개발 (2023.03 – 2024.10)

- Skylife 영화 추천 시스템: pgvector 기반 2단계 추천(메타 필터 → 벡터 유사도), 석사 연구의 첫 실서비스 적용
- 부산관광공사 여행 스케줄링 챗봇: 자연어 기반 여행지 추천 + 일정 구성 (현재 Visit Busan 상용 운영 중)

Side Project

- 서울 상권분석 시스템: 서울 Open API 8개 자동 수집 파이프라인 + LangGraph 기반 자연어 상권 분석 Agent, 9개 테이블 동적 검색, 수집부터 분석까지 전체 1인 개발

AI 서비스 개발 | (주)AIO2O 2023.03 – 2024.10 (1년 7개월)

Skylife 영화 추천 시스템 · 2023.06 ~ 2023.12 **역할**: 벡터 DB 구축, 추천 알고리즘 개발

- 장르, 배우 같은 메타 필터만으로 추천하면 비슷한 결과만 반복되는 문제가 있었음.
- 영화 메타 정보(장르, 내용 요약, 감독, 배우)를 임베딩 벡터로 만들어 PostgreSQL + pgvector에 저장하고, 추천을 2단계로 나눠 — 메타 필터링(장르, 감독, 배우 조건)으로 후보를 줄인 뒤, 영화 설명 벡터 유사도로 순위를 매기는 방식.
- 석사 때 연구한 임베딩-검색 파이프라인을 처음으로 실서비스에 적용한 프로젝트.
- 기술 스택: Python, PostgreSQL, pgvector, Embedding Models

부산관광공사 여행 스케줄링 챗봇 · 2024.01 ~ 2024.10 **역할**: 벡터 DB 구축, 추천 알고리즘 개발

- 기존 여행 검색은 "부산 / 1박 2일 / 맛집" 같은 고정 필터 수준이었고, "아이랑 갈 만한 조용한 곳" 같은 자연어 질의엔 대응이 안 됐음.
- 영화 추천에서 쓴 2단계 알고리즘(메타 필터 → 벡터 유사도)을 여행 도메인에 맞게 적용. 메타 필터링은 지역·속박 일수·카테고리, 벡터 유사도는 여행지 설명·후기·활용 정보로 구성. 추천된 관광지를 바탕으로 일정 구성까지 한 번에 처리.
- 현재 부산관광공사([Visit Busan](#))에서 상용 서비스로 운영 중.
- 기술 스택: Python, Embedding Models, Vector DB, NLP

AI 서비스 개발 | (주)플랜티넷 (Plantynet) 2024.10 – 현재 (1년 4개월, 재직 중)

vLLM 기반 텍스트 처리 API 서비스 (AI_LLM_API) · 2024.10 ~ 2025.04 **역할:** LLM 서빙 및 API 개발 담당

- 기사 요약, 분류, 태깅을 각각 따로 처리하고 있었고, 외부 API에 의존하니 비용과 속도가 계속 문제였음.
- vLLM 엔진을 FastAPI 프로세스 안에서 직접 돌리는 in-process 방식을 택함. 네트워크 왕복이 사라지고, 토큰 레벨 연속 배칭으로 동시 요청도 효율적으로 처리됨. asyncio.Semaphore로 동시 연산 슬롯 수를 제한해서 GPU 메모리 초과를 방지함.
- 한번의 추론만으로 처리하기 어려운 태스크를 개선하기 위해 LangGraph로 용도별 에이전트를 분리. 요약 (SummarizationAgent)은 목표 길이 70~130% 안에 들어올 때까지 적응형 재시도, 긴 문서 축약 (ReduceAgent)은 Map-Refine으로 불릿 추출 후 평문 변환. 키워드 추출, 다중 기사 통합 요약도 각각 별도 에이전트로 분리. API 버전을 v0/v1으로 나눠서 기존 연동을 깨지 않고 점진 전환함.
- 6종 API 엔드포인트(기사 요약, 긴문서 축약, 주제 분류, 분류체계 분석, 키워드/태그 추출, 자유 대화)를 단일 서비스로 통합. Gradio UI도 붙여서 기획팀이 바로 써볼 수 있게 구현함.
- 기술 스택: Python, FastAPI, vLLM, LangGraph, PyTorch, Gradio

Milvus 기반 Agentic RAG 시스템 (AI_PROMPT + RAG-Chatbot) · 2025.04 ~ 2025.12 **역할:** 데이터 전처리, 벡터 DB 구현, 검색 기능 구현

- 한국어 매거진 기사에 대한 질의응답이 필요했는데, 키워드 검색으로는 "여름에 읽기 좋은 글" 같은 의미 기반 질문에 대응이 안 됐음. 생성형 AI 응답에 대한 안전성 검증 체계도 없었음.
- Dense 벡터(BGE-M3, 1024차원)와 BM25 Sparse 검색을 합친 하이브리드 검색을 설계함. Dense는 의미를 잡는데 고유명사에 약하고, BM25는 그 반대라 상호보완적으로 활용. 검색 결과는 BGE-Reranker-v2-M3로 재정렬하고, 청크 앞뒤로 문맥을 자동 확장하는 컨텍스트 윈도우를 넣어 맥락 보존.
- 에이전트 흐름은 Safety Guard → Reasoning → Tool Calling → Response 순서. 입력은 LlamaGuard가 먼저 검사하고 출력도 한번 더 필터링. SSE로 토큰 단위 스트리밍, Thread 기반 Checkpoint Store로 대화 이력 관리함.
- 전처리에서 100자 이하 청크를 걸러냄(대부분 무의미한 문서). 특수문자 제거, KoNLpy 기반 토큰 검증까지 적용. 체크포인트를 MongoDB/PostgreSQL/SQLite 세 가지로 지원해서 운영 환경에 맞춰 선택 가능.
- 기술 스택: Python, LangGraph, LangChain, FastAPI, Milvus, Streamlit, BGE-M3, BGE-Reranker, MongoDB, PostgreSQL

ReAct + Reflexion 자기 개선형 AI 에이전트 (MCP Host) · 2025.09 ~ 현재 **역할:** Agent 로직 담당

- 에이전트가 Tool Calling에 실패하면 같은 실수를 반복하거나 멈춰버리는 문제가 있었음. ReAct(Yao et al., 2023) 논문의 Reasoning + Acting 루프를 기반으로 하되, 실패 시 자기 반성(Reflexion)까지 추가한 구조를 설계함.
- LangGraph StateGraph에 Actor, Tool Executor, Evaluator, Reflection 4개 노드를 구성. Actor가 Thought → Action → Observation으로 추론하고, Evaluator가 여러 키워드 규칙 + LLM 정밀 판단 2단계로 평가. FAIL이면 Reflection 노드가 원인을 분석해서 problem/solution 쌍으로 장기 기억(LessonsStore)에 저장하고, 다음에 비슷한 상황이 오면 Actor가 이 교훈을 꺼내 씀. 같은 실패가 계속되면 Early Stopping으로 중단.
- Atelier 프론트엔드의 Product → Thread → Feed 계층 구조와 호환되는 17개 도구(CRUD + Utility)를 Tool Calling으로 실행. LLM은 Gemini, OpenAI, Anthropic 사이에서 YAML 설정만 바꾸면 전환 가능.
- 실패를 기록하고 재활용하는 구조가 돌아가면서, 같은 유형의 작업에서 재시도 횟수가 줄어드는 걸 확인함.
- 기술 스택: Python, LangGraph, LangChain, Google Gemini, OpenAI, Anthropic, Pydantic, YAML

Side Project

서울 상권분석 시스템 (sbiz_db + sbiz_llm) · 2025.06 ~ 현재 **역할:** DB 스키마 설계, LLM 구현, 프롬프트 설계, 백엔드/프론트엔드 전체 개발

- 서울시 상권 데이터가 공개되어 있긴 한데, API 8개에 테이블 9개가 흩어져 있어서 SQL 모르면 사실상 못 쓰는 상황. 자연어로 "강남역 카페 매출 추이" 같은 질문을 던지면 알아서 분석해주는 서비스가 필요하다 판단함.
 - 수집 파이프라인(sbiz_db)과 분석 에이전트(sbiz_llm)를 분리해서 구현. 수집 쪽은 서울 Open API 8개 서비스의 HTML을 APISpec 클래스로 파싱해서 스키마를 자동 추출하고, pandas로 컬럼 매핑(한글 → 영문) 후 PK 기반 UPSERT로 PostgreSQL에 삽입. CLI(--sync-all, --validate 등)와 크론잡 지원.
 - 분석 에이전트는 LangGraph 기반. 쿼리가 들어오면 라우터가 rag/chat/reject로 분기하고, 상권 질문이면 Tool Calling으로 9개 테이블(추정 매출, 점포, 유동 인구, 상주 인구, 직장 인구, 임대료, 상권 변화 지표, 집객 시설 등)에 PostgreSQL 동적 검색을 진행. Tool Input은 Pydantic BaseModel, DB 접근은 asyncpg 비동기.
 - 기술 스택: Python, LangGraph, FastAPI, PostgreSQL, asyncpg, pandas, BeautifulSoup4, Docker
-

연구 개발 자료

- [개발 연구자료 정리 저서 \(Wikidocs\)](#)
 - [Notion 연구 개발 노트](#)
-

연락처

- 이메일: cyon13022@gmail.com